

MLA0303_RL_PROGRAM_9.py

```
"""
Experiment 9: TD(0), SARSA and Q-Learning algorithms for a warehouse
robot to optimize navigation and obstacle avoidance.
"""

import random
from collections import defaultdict

GRID = 5
OBSTACLES = {(1, 1), (2, 1), (3, 3)}
GOAL = (4, 4)
ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]
DELTA = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}
ALPHA, GAMMA, EPSILON, EPISODES = 0.1, 0.95, 0.15, 800

def step(state, action):
    dr, dc = DELTA[action]
    nxt = (state[0] + dr, state[1] + dc)
    if not (0 <= nxt[0] < GRID and 0 <= nxt[1] < GRID) or nxt in OBSTACLES:
        return state, -5, False
    if nxt == GOAL:
        return nxt, 20, True
    return nxt, -1, False

def epsilon_greedy(Q, state, epsilon):
    if random.random() < epsilon:
        return random.choice(ACTIONS)
    return max(ACTIONS, key=lambda a: Q[state][a])

def td0_prediction(policy_Q, episodes=EPISODES):
    """TD(0) prediction of state-value function under an epsilon-greedy policy."""
    V = defaultdict(float)
    for _ in range(episodes):
        state = (0, 0)
        for _ in range(100):
            action = epsilon_greedy(policy_Q, state, EPSILON)
            nxt, r, done = step(state, action)
            V[state] += ALPHA * (r + GAMMA * V[nxt] - V[state])
            state = nxt
            if done:
                break
    return V

def sarsa(episodes=EPISODES):
    Q = defaultdict(lambda: {a: 0.0 for a in ACTIONS})
    for _ in range(episodes):
        state = (0, 0)
        action = epsilon_greedy(Q, state, EPSILON)
        for _ in range(100):
            nxt, r, done = step(state, action)
            next_action = epsilon_greedy(Q, nxt, EPSILON)
            Q[state][action] += ALPHA * (r + GAMMA * Q[nxt][next_action] - Q[state][action])
            state, action = nxt, next_action
            if done:
                break
    return Q

def q_learning(episodes=EPISODES):
    Q = defaultdict(lambda: {a: 0.0 for a in ACTIONS})
    for _ in range(episodes):
        state = (0, 0)
        for _ in range(100):
            action = epsilon_greedy(Q, state, EPSILON)
            nxt, r, done = step(state, action)
            best_next = max(Q[nxt].values())
            Q[state][action] += ALPHA * (r + GAMMA * best_next - Q[state][action])
            state = nxt
            if done:
                break
    return Q

def greedy_path(Q):
    state, path = (0, 0), [(0, 0)]
    for _ in range(30):
        action = max(ACTIONS, key=lambda a: Q[state][a])
        state, _, done = step(state, action)
        path.append(state)
        if done:
            break
    return path

if __name__ == "__main__":
    random.seed(2)
    Q_sarsa = sarsa()
    random.seed(2)
    Q_qlearn = q_learning()
    V_td0 = td0_prediction(Q_qlearn, episodes=300)

    print("TD(0) estimated value of start state (0,0):", round(V_td0[(0, 0)], 2))
    print("SARSA learned path      :", greedy_path(Q_sarsa))
    print("Q-Learning learned path:", greedy_path(Q_qlearn))
```

PROGRAM OUTPUT

TD(0) estimated value of start state (0,0): 3.87

SARSA learned path : [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)]

Q-Learning learned path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (4, 2), (4, 3), (4, 4)]